

20250723日报

今日工作内容

1. 删除重复代码：将四种类型的setup函数中重复代码用commonsetup.go表示；
commonsetup.go如下：

```
1 // loadStandbyConfigs 加载备用配置
2 func loadStandbyConfigs(directoryName string) (map[string]string, error) {
3     configMap := make(map[string]string)
4     configDir := filepath.Join(standbyrainbowDir, directoryName)
5     // 递归遍历目录并加载有效配置文件
6     if err := filepath.Walk(configDir, func(path string, info os.FileInfo, err error)
7         error {
8             if err != nil {
9                 return fmt.Errorf("access path %s failed: %w", path, err)
10            }
11            if skipFile(info) {
12                return nil // 跳过无效文件，只会读时间戳目录下的真实文件
13            }
14            log.Info("Loading config from file:", path)
15            data, err := os.ReadFile(path)
16            if err != nil {
17                return fmt.Errorf("read file %s failed: %w", path, err)
18            }
19            configMap[filepath.Base(path)] = string(data)
20            return nil
21        }); err != nil {
22            return configMap, fmt.Errorf("traverse directory failed: %w", err) // 返回已
23            加载的部分配置
24        }
25        return configMap, nil
26    }
27
28 // skipFile 判断是否需要跳过文件
29 func skipFile(info os.FileInfo) bool {
30     return info.IsDir() || info.Size() == 0 || (info.Mode()&os.ModeSymlink) != 0
31 }
32
33 // registerConfiguration 注册配置到KV系统
```

2. 新增config.rainbow下的provider注册，以便于可以在其他代码位置直接通过provider进行Read()、Watch()、Name()的调用（这是为了备用方案与原始trpc-go的七彩石插件相统一），其代码如下：

```
1  // Provider 七彩石的DataProvider实现
2  type Provider struct {
3      kv    *KV
4      recv  chan standbyResponse
5      cache map[string]bool
6      mutex sync.Mutex
7      watch chan config.ProviderCallback
8  }
9
10 // NewProvider 初始化
11 func NewProvider(kv *KV) config.DataProvider {
12     p := &Provider{
13         kv:    kv,
14         recv:  make(chan standbyResponse, channelSize),
15         watch: make(chan config.ProviderCallback, channelSize),
16         cache: make(map[string]bool),
17     }
18     go p.loop()
19     return p
20 }
21
22 // Read 读取指定key的配置
23 func (p *Provider) Read(path string) ([]byte, error) {
24     r, err := p.kv.Get(context.TODO(), path)
25     if err != nil {
26         return nil, fmt.Errorf("rainbow provider read: kv.Get err: %w", err)
27     }
28     go p.watchPath(path)
29     return []byte(r.Value()), nil
30 }
31
32 // watchPath 监听path
33 func (p *Provider) watchPath(path string) {
```

3. 增加go协程的异常捕获。

问题：为什么要给go协程增加异常捕获呢？

原因：子协程如果未捕获panic的话会影响主协程的服务的运行，比如主协程因等待子协程而阻塞（sync.WaitGroup或chan），且子协程panic导致未释放资源，那么此时主协程可能永久阻塞。当捕获了子协程的panic后，此时可以避免子协程崩溃，使主服务不受影响。

主要代码如下：

```
1 // 协程级异常捕获
2 if r := recover(); r != nil {
3     log.Errorf("Panic in handleFileEvents: %v", r)
4     metrics.Counter("trpc.PanicNum").Incr()
5 }
```

4. 在StandbyRainbow 为true和为false的时候，分别加一下metrics上报。为false的时候上报，是看有哪些服务有接入；为true的时候上报，是为了加上告警。两种情况都是每分钟上报一次，主要代码如下：

```
1 standrainbowisfalseReport := func() {
2     metrics.IncrCounter("config-rainbow: StandbyRainbow: false", 1)
3 }
4 // 每分钟上报一次
5 go StartReporter(standrainbowisfalseReport, time.Minute)
6
7 ...
8
9 // StartReporter 启动上报
10 func StartReporter(reportFn ReporterFunc, interval time.Duration) {
11     ticker := time.NewTicker(interval)
12     defer ticker.Stop()
13     reportFn()
14     for range ticker.C {
15         reportFn()
16     }
17 }
```

明日待办：

1. 继续跟进七彩石容灾的代码修改；
2. 后续需求看强哥安排；